

Logistic Regression

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

```
data = pd.read_csv("heartfailure.csv")
```

```
data.head()
```

	age	anaemia	creatinine_phosphokinase	diabetes	ejection_fraction	high_blood_pressure	platelets	serum_creatinine	se
0	75.0	0	582	0	20	1	265000.00	1.9	
1	55.0	0	7861	0	38	0	263358.03	1.1	
2	65.0	0	146	0	20	0	162000.00	1.3	
3	50.0	1	111	0	20	0	210000.00	1.9	
4	65.0	1	160	1	20	0	327000.00	2.7	

```
X = data.drop("DEATH_EVENT", axis=1)
y = data["DEATH_EVENT"]
```

```
X.shape
```

```
(299, 12)
```

```
y.shape
```

```
(299,)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.4, random_state=0
)
```

```
X_train.shape
```

```
(179, 12)
```

```
X_test.shape
```

```
(120, 12)
```

```
model = LogisticRegression()
```

```
model.fit(X_train,y_train)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.
```

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

```
    LogisticRegression
```

```
LogisticRegression()
```

```
y_predict = model.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_predict)
```

```
print("Accuracy: ",accuracy)
```

```
Accuracy: 0.7916666666666666
```

Decision Tree

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
```

```
data = pd.read_csv("heartfailure.csv")
```

```
data.head()
```

	age	anaemia	creatinine_phosphokinase	diabetes	ejection_fraction	high_blood_pressure	platelets	serum_creatinine	se
0	75.0	0	582	0	20	1	265000.00	1.9	
1	55.0	0	7861	0	38	0	263358.03	1.1	
2	65.0	0	146	0	20	0	162000.00	1.3	
3	50.0	1	111	0	20	0	210000.00	1.9	
4	65.0	1	160	1	20	0	327000.00	2.7	

```
X = data.drop("DEATH_EVENT", axis=1)
y = data["DEATH_EVENT"]
```

```
X.shape
```

```
(299, 12)
```

```
y.shape
```

```
(299,)
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.4, random_state=0
)
```

```
X_train.shape
```

```
(179, 12)
```

```
X_test.shape
```

```
(120, 12)
```

```
model1 = DecisionTreeClassifier()
```

```
model1.fit(X_train,y_train)
```

```
DecisionTreeClassifier
```

```
y_predict = model1.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_predict)
```

```
print("Accuracy: ",accuracy)
```

```
Accuracy: 0.775
```

Random Forest

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.4)
```

```
model2 = RandomForestClassifier()
```

```
model2.fit(X_train, y_train)
```

```
▼ RandomForestClassifier ⓘ ?  
RandomForestClassifier()
```

```
y_predict = model2.predict(X_test)
```

```
accuracy = accuracy_score(y_test, y_predict)
```

```
print("Accuracy: ",accuracy)
```

```
Accuracy:  0.825
```